

Annotated Exam Solutions for Digital Electronics 2

By Baldur G.T.

2021-2023

Contents

License	3
Exam 2021	1
Problem 1	1
Solution	1
Problem 2	2
Solution	2
Problem 3	3
Solution	3
Problem 4	5
Solution	5
Problem 5	6
Solution	6
Exam 2022	7
Problem 1	7
Solution	7
Problem 2	9
solution	9
Problem 3	10
Solution	10
Problem 4	11
Solution	11
Problem 5	12
Solution	12
Exam 2023	13
Problem 1	13
Solution	13
Problem 2	14
Solution	15
Problem 3	15
Solution	15
problem 4	16
Solution	16

problem 5	17
Solution	17

License

This work is licensed under the **Creative Commons Attribution–NonCommercial 4.0 International License (CC BY-NC 4.0)**.

You are free to:

- Share — copy and redistribute the material
- Adapt — remix, transform, and build upon the material

Under the following conditions:

- **Attribution** — Appropriate credit must be given to the original source.
- **NonCommercial** — The material may not be used for commercial purposes.

For the full license text, see:

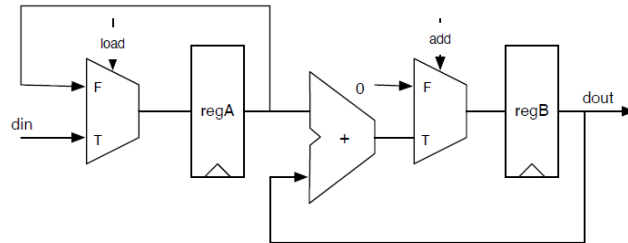
<https://creativecommons.org/licenses/by-nc/4.0/>

For Overleaf LaTeX project go to:

<https://github.com/BaldurGT/Digital-Elektronik-2-course-02139/>

Exam 2021

Problem 1



Write the Chisel code for the circuit in Figure 1. The register shall be 8 bits wide and shall be initialized to 0 on reset. Explain in your own words what the circuit does.

Solution

```
import chisel3._

class Circuit extends Module {
  val io = IO(new Bundle {
    val din  = Input(UInt(8.W))
    val load = Input(Bool())
    val add  = Input(Bool())
    val dout = Output(UInt(8.W))
  })

  // Registers initialized to 0
  val regA = RegInit(0.U(8.W))
  val regB = RegInit(0.U(8.W))

  // regA load behavior
  regA := Mux(io.load, io.din, regA)

  // Adder
  val sum = regA + regB

  // regB behavior
  regB := Mux(io.add, sum, 0.U)

  // Output
  io.dout := regB
}
```

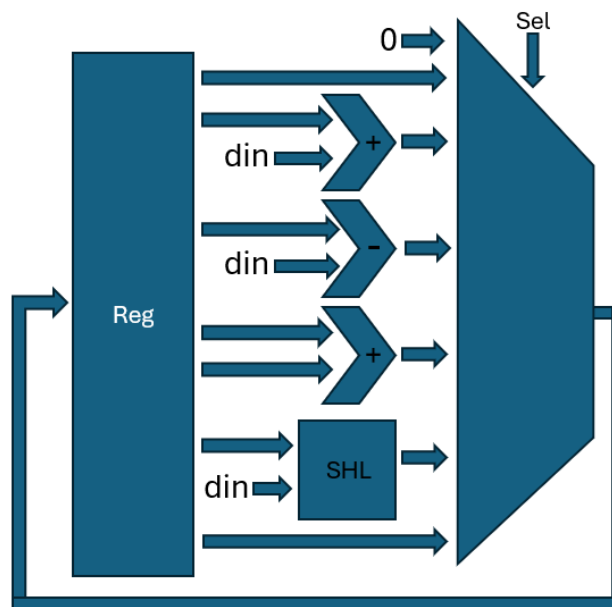
Problem 2

Given following Chisel code:

```
val reg = RegInit(0.U(8.W))
switch(sel) {
  is(0.U) { reg := 0.U}
  is(1.U) { reg := reg}
  is(2.U) { reg := reg << din}
  is(3.U) { reg := reg + din}
  is(4.U) { reg := reg - din}
  is(5.U) { reg := reg + reg}
}
```

Explain the function and draw a block diagram.

Solution



This circuit consists of a register connected to a MUX controlled by a select signal Sel. This allows for the operations:

- Set Reg = 0.
- Set Reg = Reg.
- Set Reg = Reg + din.
- Set Reg = Reg - din.
- Set Reg = Reg + Reg.
- And left-shift Reg by din bits.

Problem 3

To store state in a digital circuit we need a form of memory. In an ASIC and in an FPGA there are two types of memories: flip-flops and static RAM (SRAM) blocks. What is the cost in number of transistors for an SRAM cell and a flip-flop? Find the (English) Wikipedia entries of SRAM and a master-slave D flip-flop and explore the schematics. How many transistors are needed to implement CMOS SRAM cell or a master-slave D flip-flop? (remember, a NAND or NOR gate needs 4 transistors, an inverter 2 transistors)

Solution

Note. *This solution was provided by ChatGTP, not at calculations/solutions might be correct.*

To compare the implementation cost of memory in digital circuits, we count the number of transistors required for a CMOS SRAM cell and a master-slave D flip-flop.

SRAM Cell

*A standard CMOS SRAM bit cell is implemented as a **6-transistor (6T) cell**. It consists of:*

- *Two cross-coupled inverters used to store the bit*
- *Two access transistors used for read/write access*

Since an inverter requires 2 transistors:

$$2 \times 2 = 4$$

transistors are needed for the two inverters.

Additionally, there are two access transistors:

$$4 + 2 = 6$$

Thus, a CMOS SRAM cell requires:

$6 \text{ transistors per bit}$

Master-Slave D Flip-Flop

A master-slave D flip-flop consists of two latches:

$$\text{Master latch} + \text{Slave latch}$$

Each latch can be implemented using:

- *Two inverters*
- *Two transmission gates*

Since:

$$1 \text{ inverter} = 2 \text{ transistors}$$

and

$$1 \text{ transmission gate} = 2 \text{ transistors}$$

the transistor count for one latch is:

$$(2 \times 2) + (2 \times 2) = 8$$

Since a master-slave flip-flop uses two latches:

$$2 \times 8 = 16$$

Thus, a master-slave D flip-flop requires:

$16 \text{ transistors per bit}$

Conclusion

Comparing the two memory implementations:

$$\text{SRAM cell} = 6 \text{ transistors/bit}$$

$$\text{Master-slave D flip-flop} = 16 \text{ transistors/bit}$$

SRAM is therefore significantly more area-efficient, which is why large memories are implemented using SRAM cells, while flip-flops are mainly used for registers and sequential logic.

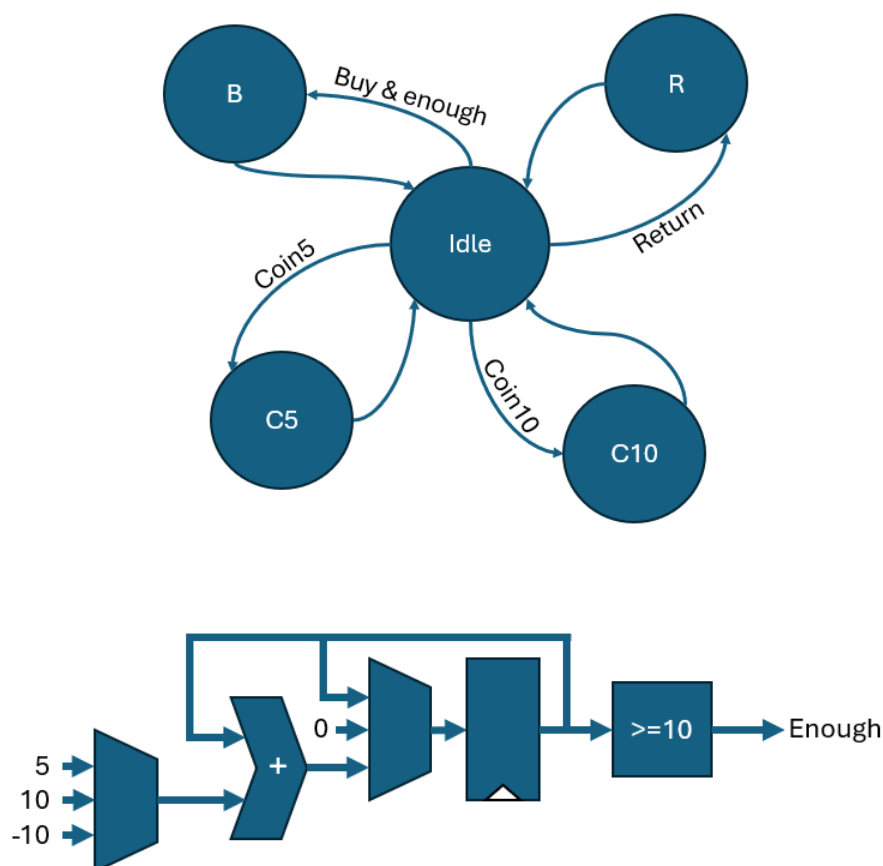
Problem 4

Draw the state machine and datapath for the following circuit:

- A vending machine that accepts coins of 5 and 10 kr.
- A soda costs 10 kr. When the sum of coins inserted into the vending machine is larger or equal to 10 kr.
- One can buy a soda by pressing the buy button.
- The inputs to the circuit are coin5, coin10, buy, and return.
- By pressing the return button, any money in the machine will be returned to the customer.
- Assume that all input signals are valid for one clock cycle (no edged detection needed).
- The outputs are a release and a moneyReturn signal.
- The output signals have to be valid for only one clock cycle.

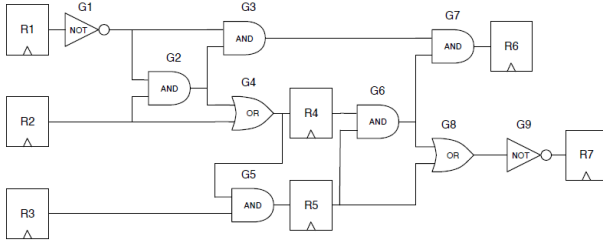
Note, that this exercise is related to your vending machine project, but it is NOT your vending machine. The specification is different (considerably simpler)

Solution



Problem 5

Find and describe the critical path in the circuit shown in the figure. What is the maximum clock frequency?



	Parameter		
DFF	setup time:	t_{SU}	3 ps
	hold time:	t_H	1 ps
	propagation delay:	$t_{cQ,prop}$	11 ps
	contamination delay:	$t_{cQ,cont}$	4 ps
Gate	propagation delay:	t_{prop}	7 ps
	contamination delay:	t_{cont}	3 ps

Solution

The critical path is the longest register-register path. Looking at the circuit we can find the longest path to be through, R1-G1-G2-G3-G7-R6 and equally as long, R1-G1-G2-G4-G5-R5.

Calculating the time we get, DFF clock-to-Q delay + 4 gate delays + setup time. With $t_{cQ,prop} = 11\text{ps}$, gate prop = 7ps, $t_{SU} = 3\text{ps}$.

$$11 \text{ ps} + 4 \cdot 7 \text{ ps} + 3 \text{ ps} = 42 \text{ ps} \quad (1)$$

From this we can find the maximum clock frequency with:

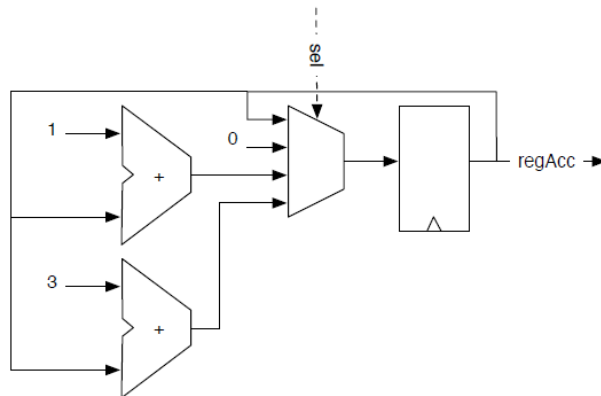
$$f_{\max} = \frac{1}{T_{\min}} = \frac{1}{42 \cdot 10^{-12} \text{ s}} = 23.8 \text{ GHz} \quad (2)$$

Exam 2022

Problem 1

Sketch the Chisel code for the circuit in the figure. The register shall be 8 bits wide and shall be initialized to 0 on reset.

Explain in your own words what the circuit does.



Solution

```
import chisel3._
import chisel3.util._

class Circuit extends Module {
  val io = IO(new Bundle {
    val sel = Input(UInt(2.W))
    val regAcc = Output(UInt(8.W))
  })

  val regAcc = RegInit(0.U(8.W))

  // Adders
  val plus1 = regAcc + 1.U
  val plus3 = regAcc + 3.U

  val muxVal = MuxLookup(io.sel, regAcc)(
    Seq(
      0.U -> regAcc,
      1.U -> 0.U,
      2.U -> plus1,
      3.U -> plus3
    )
  )

  regAcc := muxVal
  io.regAcc := regAcc
}
```

```
import chisel3._
import chisel3.util._

class Circuit extends Module {
  val io = IO(new Bundle {
    val sel = Input(UInt(2.W))
    val regAcc = Output(UInt(8.W))
  })

  val regAcc = RegInit(0.U(8.W))

  switch(sel) {
    is(0.U) {regAcc := regAcc}
    is(1.U) {regAcc := 0}
    is(2.U) {regAcc := regAcc + 1.U}
    is(3.U) {regAcc := regAcc + 3.U}
  }

  io.regAcc := regAcc
}
```

The system consists of an register than through the select signal Sel, can be set to 0, or increment by +1, or +3.

The output is the stored value of the register.

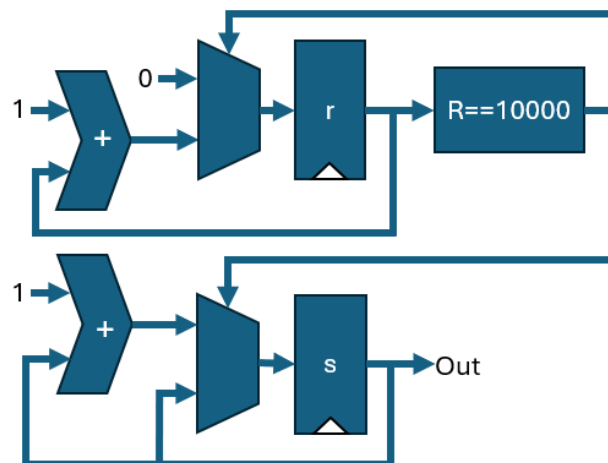
Problem 2

Given following Chisel code:

```
class X extends Module {  
  val io = IO(new Bundle{  
    val o = Output(UInt(32.W))  
  })  
  val r = RegInit(0.U(32.W))  
  val s = RegInit(0.U(32.W))  
  r := r + 1.U  
  when (r === 10000.U) {  
    r := 0.U  
    s := s + 1.U  
  }  
  io.o := s  
}
```

Explain the function and draw a block diagram.

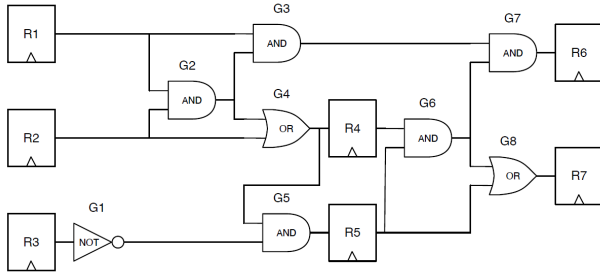
solution



This system consists of two registers, r and s. The r-register starts at 0 and counts to 10000 before resetting to 0.

When the r-register resets to 0, the s-register increments by 1, the output is the value stored by the s-register.

Problem 3



	Parameter		
DFF	setup time:	t_{SU}	3 ps
	hold time:	t_H	1 ps
	propagation delay:	$t_{cQ,prop}$	11 ps
	contamination delay:	$t_{cQ,cont}$	4 ps
Gate	propagation delay:	t_{prop}	7 ps
	contamination delay:	t_{cont}	3 ps

Find and describe the critical path in the circuit shown in the figure. What is the maximum clock frequency?

Solution

There are 4 routes, that are equally long:

R1-G2-G4-G5-R5

R1-G2-G3-G7-R6

R2-G2-G4-G5-R5

Rr-G2-G3-G7-R6

Calculating the time we get, DFF clock-to-Q delay + 4 gate delays + setup time. With $t_{cQ,prop} = 11\text{ps}$, gate prop = 7ps, $t_{SU} = 3\text{ps}$.

$$11 \text{ ps} + 3 \cdot 7 \text{ ps} + 3 \text{ ps} = 35 \text{ ps} \quad (3)$$

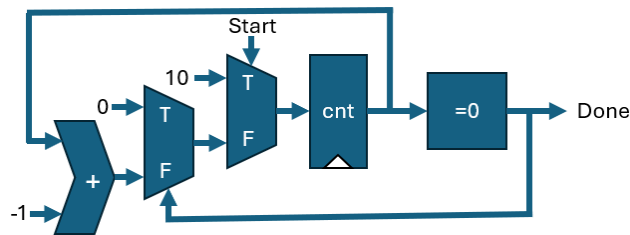
From this we can find the maximum clock frequency with:

$$f_{\max} = \frac{1}{T_{\min}} = \frac{1}{35 \cdot 10^{-12} \text{ s}} = 28.6 \text{ GHz} \quad (4)$$

Problem 4

Implement a timer. It has one input: start and one output: done. The timer is started with a single clock cycle high (1) of input start. It shall count for 10 clock cycles and then output high (1) at output done. During counting done shall be low (0). Draw the schematic of the circuit and sketch the Chisel code to describe it.

Solution



```
import chisel3._
import chisel3.util._

class Circuit extends Module {
  val io = IO(new Bundle {
    val start = Input(Bool())
    val done = Output(Bool())
  })

  val cnt = RegInit(0.U(4.W))

  val done = cnt === 0.U
  val dec = cnt - 1.U

  cnt := Mux(io.start,
    10.U,
    Mux(done,
      0.U,
      dec
    )
  )

  io.done := done
}
```

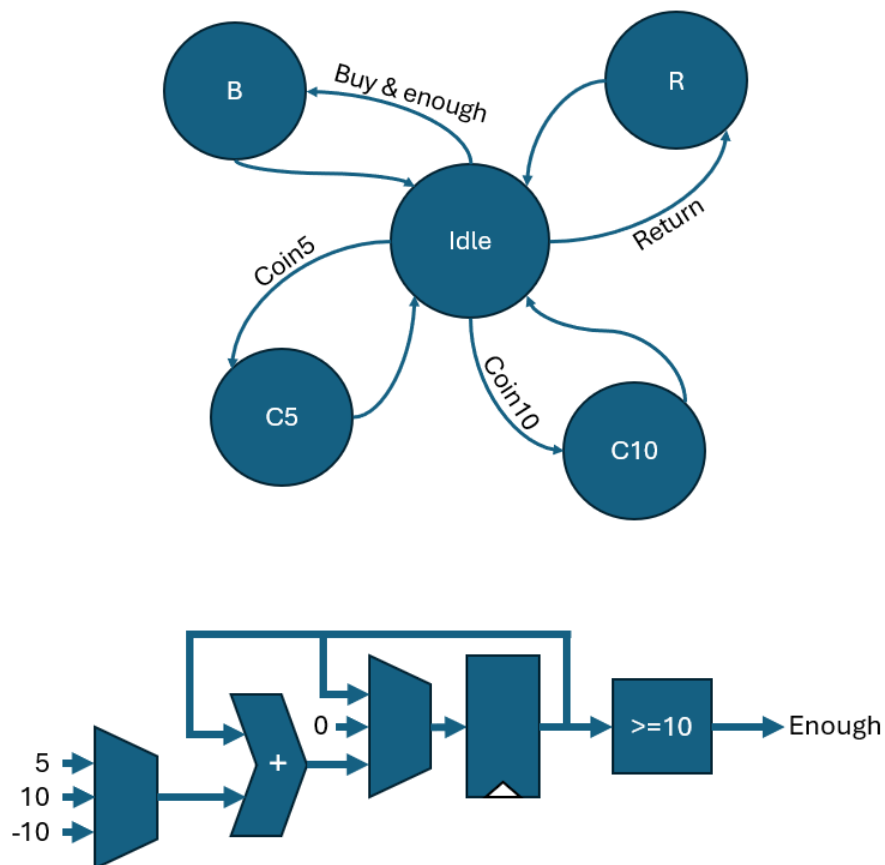
Problem 5

Draw the state machine and the datapath for the following circuit:

- A vending machine that accepts coins of 5 and 10 kr. A soda costs 10 kr.
- When the sum of coins inserted into the vending machine is larger or equal to 10 kr, one can buy a soda by pressing the buy button.
- The inputs to the circuit are coin5, coin10, buy, and return.
- By pressing the return button, any money in the machine will be returned to the customer.
- Assume that all input signals are valid for one clock cycle (no edged detection needed).
- The outputs are a release and a money Return signal. The output signals have to be valid for only one clock cycle.

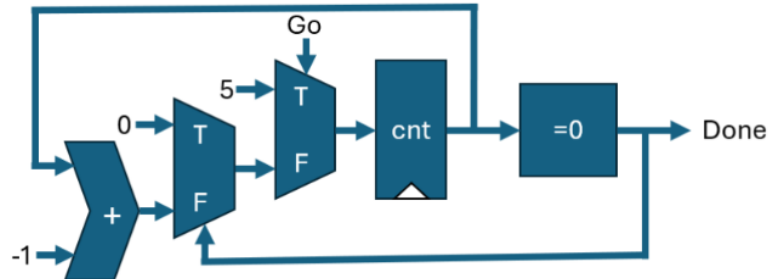
Note, that this exercise is related to your vending machine project, but it is NOT your vending machine. The specification is different (considerably simpler).

Solution



Exam 2023

Problem 1



Sketch the Chisel code for the circuit in the figure. The register shall be 10 bits wide and shall be initialized to 0 on reset. Explain in your own words what the circuit does.

Solution

```
import chisel3._
import chisel3.util._

class Circuit extends Module {
  val io = IO(new Bundle {
    val go = Input(Bool())
    val done = Output(Bool())
  })

  // Inint cnt starting at 0, and 10 bit wide.
  val cnt = RegInit(0.U(10.W))

  // Set done high, unless counter is 0.
  val done = cnt === 0.U

  // count 1 down.
  val dec = cnt - 1.U

  // Set cnt to 5 when go is high, else subtract one
  cnt := Mux(io.go,
    5.U,
    Mux(done,
      0.U,
      dec
    )
  )

  // Output.
  io.done := done
}
```

This circuit implements a counter, counting from 5 to 0, the current number is stored in a 10 bit wide register.

The output is a boolean, that remains low as long as the counter has not reached 0, when reach 0 the counter goes high and stop counting down.

Problem 2

Given following Chisel code:

```
import chisel3._
import chisel3.util._

class X extends Module {
  val io = IO(new Bundle{
    val in = Flipped(new DecoupledIO(UInt(8.W)))
    val out = new DecoupledIO(UInt(8.W))
  })

  val dataReg = Reg(UInt(8.W))
  val emptyReg = RegInit(true.B)

  io.in.ready := emptyReg
  io.out.valid := !emptyReg
  io.out.bits := dataReg

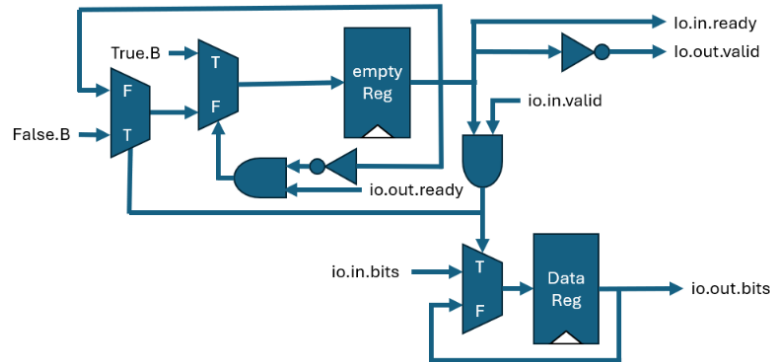
  when (emptyReg & io.in.valid) {
    dataReg := io.in.bits
    emptyReg := false.B
  }

  when (!emptyReg & io.out.ready) {
    emptyReg := true.B
  }
}

// DecoupledIO is a Chisel library class and defined as follows:
class DecoupledIO[T <: Data](gen: T) extends Bundle {
  val ready = Input(Bool())
  val valid = Output(Bool())
  val bits = Output(gen)
}
```

Explain the function and draw a block diagram.

Solution



This is a single-element Decoupled buffer, where one word is stored at a time. It is used when a sender and receiver are transferring data. The valid flag is used by the sender to indicate that valid data is available, and the ready flag is used by the receiver to indicate that it is ready to receive the data.

This way, data is only transferred when:

1. valid data is available, and
2. the receiver is ready to receive it.

dataReg is an 8-bit register used to store a data value. The stored data comes from io.in.bits, and the output data is connected to io.out.bits.

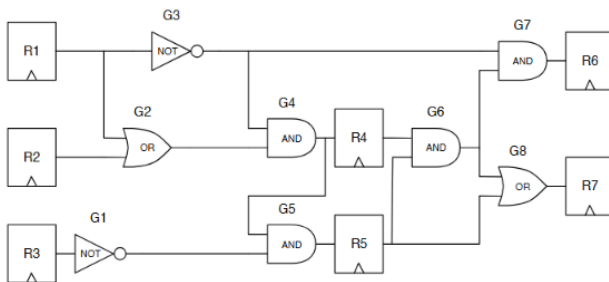
emptyReg is a Boolean register that indicates whether the buffer is empty. It is used to control the ready and valid handshake signals.

When emptyReg is true and io.in.valid is true, the module accepts input data. The value from io.in.bits is stored into dataReg, and emptyReg is set to false, indicating that the buffer now contains valid data.

When emptyReg is false and io.out.ready is true, the output data has been accepted by the receiver. The module then sets emptyReg to true, indicating that the buffer is empty again.

Problem 3

Given following circuit and timing parameters, Find and describe the critical path in the circuit. What is the maximum clock frequency?



	Parameter		
DFF	setup time:	t_{SU}	3 ps
	hold time:	t_H	1 ps
	propagation delay:	$t_{cQ,prop}$	7 ps
	contamination delay:	$t_{cQ,cont}$	4 ps
Gate	propagation delay:	t_{prop}	5 ps
	contamination delay:	t_{cont}	2 ps
Inverter	propagation delay:	$t_{prop-inv}$	4 ps
	contamination delay:	$t_{cont-inv}$	1 ps

Solution

We look at the longest path, that being R1-G3-G4-G5-R5, or R1-G2-G4-G5-R5. In number of gates, but noticing that an inverter is slightly shorter than an or gate making R1-G2-G4-G5-R5 the longest.

meaning we have a critical time of $t_{SU} + t_{Q,prop} + 3 \cdot t_{prop}$, or:

$$3ps + 7ps + 3 \cdot 5ps = 25ps \quad (5)$$

OR a frequency of $f = \frac{1}{T} = \frac{1}{25 \cdot 10^{-12}} = 40GHz$

problem 4

Assume you have inputs from a bouncing button to your circuit. The input signals are not noisy, so no filtering is needed. What circuit do you need for your input? Explain in plain text what is needed and sketch some Chisel code.

Solution

We need a debouncing circuit. One solution is subsampling the input with a low enough frequency. Chisel code:

```
import chisel3._
import chisel3.util._

class Circuit extends Module {
  val io = IO(new Bundle {
    val buttonIn  = Input(Bool())
    val buttonOut = Output(Bool())
  })

  val buttonReg = Reg(Bool())
  buttonReg := io.buttonIn
  val buttonDebReg = RegInit(false.B)

  val cnt = RegInit(0.U(32.W))
  val fac = 100000000/100
  val tick = cnt === (fac-1).U
  cnt := cnt + 1.U

  when(tick) {
    cnt := 0.U
    buttonDebReg := buttonReg
  }

  io.buttonOut := buttonDebReg
}
```

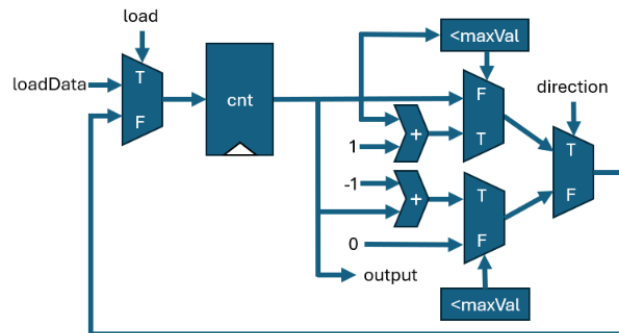
problem 5

Draw a block diagram for a saturating counter that can count up, count down, and load.

This counter, however, must saturate when counting up (at a programmable maximum count) and counting down (at zero).

Also sketch Chisel code to describe this circuit.

Solution



```
import chisel3._
import chisel3.util._

class Circuit extends Module {
  val io = IO(new Bundle {
    val direction = Input(Bool())    // true = up, false = down
    val load      = Input(Bool())
    val loadData  = Input(UInt(8.W))
    val maxVal    = Input(UInt(8.W))
    val output    = Output(UInt(8.W))
  })

  val cnt = RegInit(0.U(8.W))

  when(io.load) {
    cnt := io.loadData
  }

  // count up
  .elsewhen(io.direction) {
    when(cnt < io.maxVal) {
      cnt := cnt + 1.U
    } .otherwise {
      cnt := io.maxVal
    }
  }

  // count down
  .otherwise {
    when(cnt > 0.U) {
      cnt := cnt - 1.U
    } .otherwise {
      cnt := 0.U
    }
  }

  io.output := cnt
}
```